

JavaScript



TOPICS

- Overview of JavaScript
- Object Orientation and JavaScript
- Syntactic Characteristics
- Primitives
- Operations
- Expressions
- Screen Output and Keyboard Input
- Control Statements
- Object Creation and Modification
- Arrays
- Functions
- Constructors
- Pattern Matching

ORIGINS AND EVOLUTION OF JAVASCRIPT

- JavaScript was originally developed at Netscape by Brendan Eich in 1995

JavaScript versions timeline



JAVASCRIPT

- Java!=JavaScript
- Uses:



GENERAL SYNTACTIC CHARACTERISTICS

- JavaScript code is inserted between `<script>` and `</script>` tags when used in an HTML document
- Scripts can be placed inside the **body** or the **head** section of an HTML page or inside both the **head and body**
- JavaScript can also be used in external files. The file extension of the JavaScript file should be `.js`.
- To use an external script put the name of the script file in the `src` attribute of a script tag. External scripts cannot contain script tags.

```
<script src="myScript1.js"></script>
```

GENERAL SYNTACTIC CHARACTERISTICS

Identifiers:

- Identifiers must begin with a letter, an underscore (_), or a dollar
- Subsequent characters may be letters, underscores, dollar signs, or digits
- There is no length limitation for identifiers
- Case sensitive

Comments:

- Single line: //
- Multiline: /* */

GENERAL SYNTACTIC CHARACTERISTICS

- Reserve words:

abstract	delete	function	null	throw
boolean	do	goto	package	throws
break	double	if	private	transient
byte	else	implements	protected	true
case	enum	import	public	try
catch	export	in	return	typeof
char	extends	instanceof	short	var
class	false	int	static	void
const	final	interface	super	volatile
continue	finally	long	switch	while
debugger	float	native	synchronized	with
default	for	new	this	

- Safe way to organize JavaScript statements is to put each on its own line whenever possible and terminate each statement with a semicolon

PRIMITIVE TYPES

- JavaScript primitive types: : Number, String, Boolean, Undefined, Null
- Number, String, and Boolean have wrapper objects

In the cases of Number and String, primitive

- values and objects are coerced back and forth so that primitive values can be treated essentially as if they were objects
- All numeric values are stored in double-precision floating point
- String literals are delimited by either ' or "
- Can include escape sequences (e.g., \t)

PRIMITIVE TYPES

- Boolean values are true and false
- The only Null value is null
- The only Undefined value is undefined
- **Numeric operators:** unary +,-, ++,-- , binary +,-*,/, %

Variables:

- JavaScript is dynamically typed - The interpreter determines the type of a particular occurrence of a variable
- Variables are not typed; values are. A variable
- can have the value of any primitive type, or it can be a reference to any object.
- Variables can be either implicitly or explicitly declared
- In many cases, the interpreter converts the type of a value to whatever is needed for the context in which it appears.
- ```
var sum = 0,
 today = "Monday",
 flag = false;
```



# THE MATH OBJECT

- It provides a collection of properties of Number objects and methods that operate on Number object
- It has methods for:
- trigonometric functions, such as `sin`
- To truncate a number `floor`
- To round a number `round`
- To return the largest of two given numbers `max`
- All of the Math methods are referenced through the Math object: `Math.sin(x)`

# THE NUMBER OBJECT

- It includes a collection of useful properties that have constant values

| Property          | Meaning                                                  |
|-------------------|----------------------------------------------------------|
| MAX_VALUE         | Largest representable number on the computer being used  |
| MIN_VALUE         | Smallest representable number on the computer being used |
| NaN               | Not a number                                             |
| POSITIVE_INFINITY | Special value to represent infinity                      |
| NEGATIVE_INFINITY | Special value to represent negative infinity             |
| PI                | The value of $\pi$                                       |

- Any arithmetic operation that results in an error or that produces a value that cannot be represented as a double-precision floating-point number, such as a number that is too large returns the value “not a number,” which is displayed as **NaN**



# THE NUMBER OBJECT

- The toString method converts the number through which it is called to a string.
- Because numeric primitives and Number objects are always coerced to the other when necessary, toString can be called through a numeric primitive
- Example: `price.toString();`

# THE STRING CATENATION OPERATOR

- String catenation is specified with the operator denoted by a plus sign (+)
- Example: "hello"+"abc"



# IMPLICIT TYPE CONVERSION

- When a value of one type is used in a position that requires a value of a different type, JavaScript attempts to convert the value to the type that is required
- When used as a number, null is 0
- When used as a number, undefined is interpreted as NaN.

# EXPLICIT TYPE CONVERSION

- Number to string conversion
  - Using String() constructor
  - toString() method
- //string to number conversion
- Number constructor
- by subtracting zero from the string
- parseInt(), parseFloat()



# STRING PROPERTIES AND METHODS

- String methods can always be used through String primitive values, as if the values were objects because of coercion
- length property: gives number of characters in a string
- String Methods:

| Method      | Parameters           | Result                                                                                     |
|-------------|----------------------|--------------------------------------------------------------------------------------------|
| charAt      | A number             | Returns the character in the String object that is at the specified position               |
| indexOf     | One-character string | Returns the position in the String object of the parameter                                 |
| substring   | Two numbers          | Returns the substring of the String object from the first parameter position to the second |
| toLowerCase | None                 | Converts any uppercase letters in the string to lowercase                                  |
| toUpperCase | None                 | Converts any lowercase letters in the string to uppercase                                  |

# THE TYPEOF OPERATOR

- returns the type of its single operand
- produces "number", "string", or "boolean" if the operand is of primitive type Number, String, or Boolean, respectively
- If the operand is an object or null, typeof produces "object" => Objects do not have types
- If the operand is a variable that has not been assigned a value, typeof produces "undefined", reflecting the fact that variables themselves are not typed
- typeof operator always returns a string
- **typeof x** and **typeof(x)** are equivalent



# ASSIGNMENT STATEMENTS

- simple assignment operator =
- compound assignment operators +=

# THE DATE OBJECT

- A Date object is created with the new operator and the Date constructor
- `var today = new Date();`
  - Builds an object with the current date and time for its properties

- Date methods:

| Method                       | Returns                                                      |
|------------------------------|--------------------------------------------------------------|
| <code>toLocaleString</code>  | A string of the Date information                             |
| <code>getDate</code>         | The day of the month                                         |
| <code>getMonth</code>        | The month of the year, as a number in the range from 0 to 11 |
| <code>getDay</code>          | The day of the week, as a number in the range from 0 to 6    |
| <code>getFullYear</code>     | The year                                                     |
| <code>getTime</code>         | The number of milliseconds since January 1, 1970             |
| <code>getHours</code>        | The hour, as a number in the range from 0 to 23              |
| <code>getMinutes</code>      | The minute, as a number in the range from 0 to 59            |
| <code>getSeconds</code>      | The second, as a number in the range from 0 to 59            |
| <code>getMilliseconds</code> | The millisecond, as a number in the range from 0 to 999      |



# SCREEN OUTPUT AND KEYBOARD INPUT

- The normal output screen for JavaScript is the same as the screen in which the content of the host HTML document is displayed
- JavaScript models the HTML document with the **Document** object
- The window in which the browser displays an HTML document is modeled with the **Window** object
- The Window object includes two properties, **document** and **window**.
- The document property refers to the Document object.
- The window property is self-referential; it refers to the Window object.

# SCREEN OUTPUT AND KEYBOARD INPUT

- The Document object has several properties and methods.
- **write()**: used to create output, which is dynamically created HTML document content
- The parameter of write can include any HTML tags and content
- write often includes `<br />`
- The write method actually can take any number of parameters.
- Multiple parameters are catenated and placed in the output
- The **writeln()** method implicitly adds "\n" to its parameter, but since browsers ignore line breaks when displaying HTML, it has no effect on the output



# SCREEN OUTPUT AND KEYBOARD INPUT

- Window includes three methods that create dialog boxes for three specific kinds of user interactions
- The default object for JavaScript is the Window object currently being displayed, so calls to these methods need not include an object reference.
- Methods: alert, confirm, and prompt

# SCREEN OUTPUT AND KEYBOARD INPUT

`alert()`:

- It opens a dialog window and displays its parameter in that window.
- It also displays an OK button.
- The string parameter of alert is not HTML code; it is plain text. Therefore, the string parameter of alert may include `\n` but never should include `<br />`



# SCREEN OUTPUT AND KEYBOARD INPUT

## `confirm()`:

- It opens a dialog window in which the method displays its string parameter, along with two buttons: OK and Cancel
- `confirm` returns a Boolean value that indicates the user's button input: `true` for OK and `false` for Cancel.
- This method is often used to offer the user the choice of continuing some process
- After the user presses one of the buttons in the `confirm` dialog window, the script can test the variable and react accordingly

# SCREEN OUTPUT AND KEYBOARD INPUT

## `prompt()`:

- creates a dialog window that contains a text box used to collect a string of input from the user, which `prompt` returns as its value
- this window also includes two buttons: OK and Cancel
- `Prompt` takes two parameters: the string that prompts the user for input and a default string in case the user does not type a string before pressing one of the two buttons.
- In many cases, an empty string is used for the default input.
- `alert`, `prompt`, and `confirm` cause the browser to wait for a user response. In the case of `alert`, the OK button must be pressed for the JavaScript interpreter to continue. The `prompt` and `confirm` methods wait for either OK or Cancel to be pressed



# CONTROL STATEMENTS

## Control Expressions

- The expressions upon which statement flow control can be based
- They include primitive values, relational expressions, and compound expressions
- The result of evaluating a control expression is one of the Boolean values **true** and **false**
- If the value of a control expression is a string, it is interpreted as true unless it is empty string (" ")
- If the value is a number, it is true unless it is zero (0)
- If the special value, NaN, is interpreted as a Boolean, it is false.
- If undefined is used as a Boolean, it is false.
- When interpreted as a Boolean, null is false.
- When interpreted as a number, true has the value 1 and false has the value 0

## Relational operators:

| Operation                   | Operator |
|-----------------------------|----------|
| Is equal to                 | ==       |
| Is not equal to             | !=       |
| Is less than                | <        |
| Is greater than             | >        |
| Is less than or equal to    | <=       |
| Is greater than or equal to | >=       |
| Is strictly equal to        | ===      |
| Is strictly not equal to    | !==      |



## Strict equality vs Loose equality:

- The loose equality operator '==' allows us to compare two or more operands by converting their value to a common type first and then checking for the equality between them
  - If one operand is a string and the other is a number, Java Script attempts to convert the string to a number. If one operand is Boolean and the other is not, the Boolean value is converted to a number (1 for true, 0 for false)
- Strict equality operator '===' allows us to compare two or more operands by checking the equality between **the values as well as their types** . It returns true only if the values and the type both match with the other operand

## Operator precedence and associativity quality:

| Operators*                      | Associativity |
|---------------------------------|---------------|
| ++, --, unary -                 | Right         |
| *, /, %                         | Left          |
| +, -                            | Left          |
| >, <, >=, <=                    | Left          |
| ==, !=                          | Left          |
| ===, !==                        | Left          |
| &&                              | Left          |
|                                 | Left          |
| =, +=, -=, *=, /=, &&=,   =, %= | Right         |



## Selection Statements: if

```
if (condition1) {
 // block of code to be executed if condition1 is true
} else if (condition2) {
 // block of code to be executed if the condition1 is false and condition2 is true
} else {
 // block of code to be executed if the condition1 is false and condition2 is false
}
```

## The switch Statement

- The control expression of a switch statement could evaluate to a number, a string, or a Boolean value.
- Case labels also can be numbers, strings, or Booleans,
- Different case values can be of different types

```
switch(expression) {
 case x:
 // code block
 break;
 case y:
 // code block
 break;
 default:
 // code block
}
```



## Loop Statements

while loop:

```
while (condition) {
 // code block to be executed
}
```

do while

```
do {
 // code block to be executed
}
while (condition);
```

for loop:

```
for (expression 1; expression 2; expression 3) {
 // code block to be executed
}
```

# OBJECT CREATION AND MODIFICATION

- Objects are often created with a **new**, which must include a call to a constructor method
- constructor that is called in the **new** expression creates the properties that characterize the new object
- The constructor both creates and initializes the properties

```
var my_object = new Object();
```

- It creates an object that has no properties



# OBJECT CREATION AND MODIFICATION

```
var my_car = new Object(); // Creates an Object object
my_car.make = "Ford"; // Creates and initializes the make property
my_car.model = "Fusion"; // Creates and initializes model
```

```
var my_car = {make: "Ford", model: "Fusion"};
```

- We can create a new object that is a property of my\_car with properties of its own

```
my_car.engine = new Object();
my_car.engine.config = "V6";
my_car.engine.hp = 263;
```

# OBJECT CREATION AND MODIFICATION

Accessing properties:

- object-dot-property notation: `my_car.make`
- Array notation: `my_car["make"];`
- A property can be deleted with delete: `delete my_car.model;`
- `for-in`, can be used for listing the properties of an object  
    `for (identifier in object)`  
        statement or compound statement

**Objects are mutable:** They are addressed by reference, not by value.



# ARRAYS

- An array is an ordered list of values
- Array elements can be primitive values or references to other objects
- JavaScript arrays have dynamic lengths
- They can contain a mix of different data types

# ARRAY OBJECT CREATION

- An array object can be created in two distinct ways

## 1. Using new operator

- `var my_array = new Array(1, 2, "three", "four");`

`//Array object of length 4 is created , initialized`

- `var my_array2 = new Array(100);`

`//Array object of length 100 is created, without actually creating any elements`

- Whenever a call to the Array constructor has a single parameter, that parameter is taken to be the number of elements, not the initial value of a one-element array



# ARRAY OBJECT CREATION

## 2. Using literal array value

```
var my_array = [1, 2, "three", "four"];
```

# ARRAY OBJECT CHARACTERISTICS

- The lowest index of every JavaScript array is zero
- The length of an array is the highest subscript to which a value has been assigned, plus 1
- if `my_list` is an array with four elements and `my_list[47] = 2222;` is executed, the new length of `my_list` will be 48
- `length` property is created for every array object by the Array constructor.
- length of an array is both read and write accessible
- `my_list.length = 1002;`
- Assigning a value to the length property can lengthen, shorten, or not affect the array's length



# ARRAY OBJECT CHARACTERISTICS

- Only the assigned elements of an array actually occupy space
- If it is convenient to use the subscript range of 100 to 150, an array of length 151 can be created; But if only the elements indexed 100 to 150 are assigned values, the array will require the space of 51 elements
- The length property of an array is not necessarily the number of elements allocated
- `new_list.length = 1002;`
- sets the length property of `new_list` to 1002, but `new_list` may have no elements that have values or occupy space

# ARRAY METHODS

## join()

- The join method converts all of the elements of an array to strings and catenates them into a single string
- If no parameter is provided to join, the values in the new string are separated by commas
- If a string parameter is provided, it is used as the element separator

## reverse():

- It reverses the order of the elements of the Array object through which it is called.



# ARRAY METHODS

## sort()

- It coerces the elements of the array to become strings if they are not already strings and sorts them alphabetically

## concat()

- It catenates its actual parameters to the end of the Array object on which it is called

# ARRAY METHODS

## slice()

- It returns the part of the Array object specified by its parameters, which are used as subscripts
- The array returned has the elements of the Array object through which it is called, from the first parameter up to, but not including, the second parameter
- If slice is given just one parameter, the array that is returned has all of the elements of the object, starting with the specified index

## toString()

- each of the elements of the object is converted (if necessary) to a string. These strings are catenated, separated by commas.



# ARRAY METHODS

## pop()

- It removes an element from the end

## push()

- It inserts an element to the end

## shift()

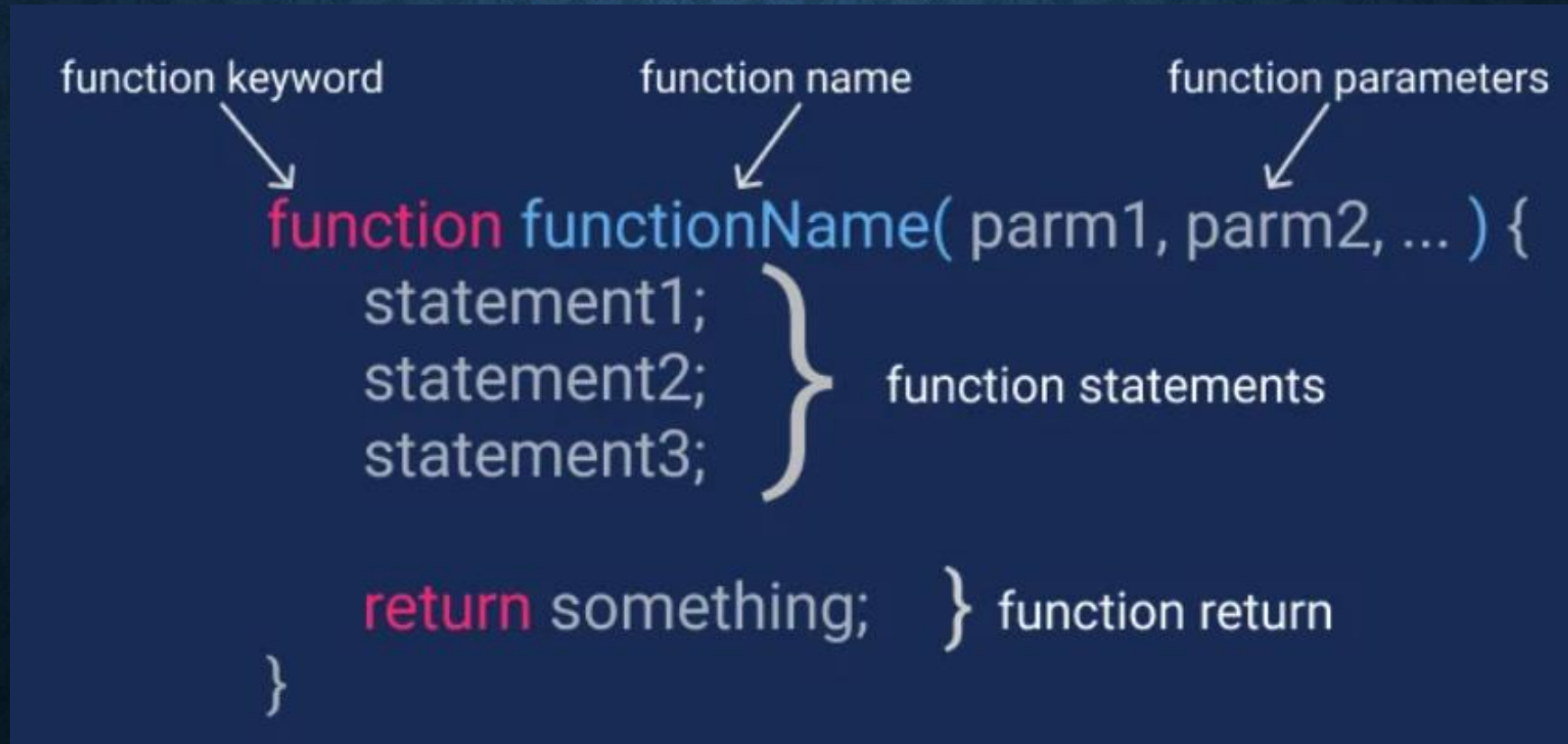
- It removes an element from the beginning of an array

## unshift()

- It adds an element to the beginning of an array

# FUNCTIONS

A JavaScript function is a block of code designed to perform a particular task





# FUNCTIONS

- A return statement returns control from the function in which it appears to the function's caller
- Optionally, it includes an expression, whose value is returned to the caller
- If there are no return statements in a function or if the specific return that is executed does not include an expression, the value returned is **undefined**
- This is also the case if execution reaches the end of the function body without executing a return statement

# FUNCTIONS

- JavaScript functions are objects, so variables that reference them can be treated as are other object references
- They can be passed as parameters, be assigned to other variables, and be the elements of an array
- The interpreter sees the definition of a function before it sees a call to the function



# FUNCTIONS-LOCAL VARIABLE

- Variables that are implicitly declared have global scope-even if the implicit declaration occurs within a function definition
- Variables that are explicitly declared outside function definitions also have global scope
- Any variable explicitly declared with var in the body of a function has local scope
- If a variable that is defined both as a local variable and as a global variable appears in a function, the local variable has precedence, effectively hiding the global variable with the same name

# FUNCTIONS-PARAMETERS

- The parameter values that appear in a call to a function are called **actual parameters**
- The parameter names that appear in the header of a function definition are called **formal parameters**
- JavaScript uses the pass-by-value parameter-passing method
- When a function is called, the values of the actual parameters specified in the call are, in effect, copied into their corresponding formal parameters, which behave exactly like local variables
- Because of JavaScript's dynamic typing, there is no type checking of parameters



# FUNCTIONS-PARAMETERS

- Because of JavaScript's dynamic typing, there is no type checking of parameters
- The number of parameters in a function call is not checked against the number of formal parameters in the called function.
- In the function, excess actual parameters that are passed are ignored; excess formal parameters are set to undefined
- All parameters are communicated through a property array, **arguments** property named **length**
- By accessing **arguments.length**, a function can determine the number of actual parameters that were passed.
- Because the arguments array is directly accessible, all actual parameters specified in the call are available, including actual parameters that do not correspond to any formal parameters

# FUNCTIONS-PARAMETERS

- All parameters are communicated through a property array, **arguments** property named **length**
- By accessing **arguments.length**, a function can determine the number of actual parameters that were passed
- Because the arguments array is directly accessible, all actual parameters specified in the call are available, including actual parameters that do not correspond to any formal parameters



## **sort() Revisited**

- sort method for array objects converts the array's elements to strings, if necessary, and then sorts them alphabetically
- If we need to sort something other than strings, or if you want an array to be sorted in some order other than alphabetically as strings, the comparison operation must be supplied to the sort method by the caller
- Such a comparison operation is passed as a parameter to sort

## sort() Revisited

- The comparison function must return a negative number if the two elements being compared are in the desired order, zero if they are equal, and a number greater than zero if they must be interchanged

```
function my_order(a,b){return a-b;}
```

```
array.sort(my_order)
```

- Rather than defining a comparison function elsewhere and passing its name, the function definition can appear as the actual parameter in the call to sort

```
array.sort(function(a, b){return a-b;})
```

- Such a function is nameless and can be used only where its definition appears.



# CLASSES AND CONSTRUCTORS

- The version of JavaScript (ES6) introduced the use of classes
- A class in javascript is basically a blueprint or template of the object; New objects can be created from a class
- JavaScript constructors are special functions that create and initialize the properties of newly created objects
- Every new expression must include a call to a constructor
- **this** variable is used to construct and initialize the properties of the object

# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- JavaScript has powerful pattern-matching capabilities based on regular expressions
- Two approaches: methods of the **RegExp** object and methods of the **String** object
- Patterns are specified in a form that is based on regular expressions
- Patterns are sent as parameters to the pattern matching methods, are delimited with slashes



# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- `search( )` takes a pattern as a parameter
- It returns the position in the String object (through which it is called) at which the pattern matched. If there is no match, search returns `-1`
- The position of the first character in the string is `0`

# CHARACTER AND CHARACTER-CLASS PATTERNS

- Metacharacters are characters that have special meanings in some contexts in patterns

\ | ( ) [ ] { } ^ \$ \* + ? .

- Metacharacters can themselves be matched by being immediately preceded by a backslash



# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- **A . matches any character except newline**
- It is often convenient to be able to specify classes of characters rather than individual characters. Such classes are defined by placing the desired characters in brackets

| Name | Equivalent Pattern | Matches                         |
|------|--------------------|---------------------------------|
| \d   | [0-9]              | A digit                         |
| \D   | [^0-9]             | Not a digit                     |
| \w   | [A-Za-z_0-9]       | A word character (alphanumeric) |
| \W   | [^A-Za-z_0-9]      | Not a word character            |
| \s   | [ \r\t\n\f]        | A white-space character         |
| \S   | [^ \r\t\n\f]       | Not a white-space character     |

# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- In many cases, it is convenient to be able to repeat a part of a pattern, often a character or character class.
- To repeat a pattern, a numeric quantifier, delimited by braces, is attached
- `/xy{4}z/` matches `xyyyyz`



# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- An asterisk(\*) matches zero or more repetitions
- A plus sign(+) matches one or more repetitions
- A question mark(?) matches one or none
- /x\*y+z?/ matches strings with any number of x's (including zero), followed by one or more y's, possibly followed by z

# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- The quantifiers are often used with the predefined character-class names
- `/\d+\.\d*/` matches a string of one or more digits followed by a decimal point and possibly more digits
- `/[A-Za-z]\w*/` matches the identifiers (a letter, followed by zero or more letters, digits, or underscores) in some programming languages



# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- It is useful to be able to specify that a pattern must match at a particular position in a string
- A pattern is tied to a string position with an anchor
- A pattern can be specified to match only at the beginning of the string by preceding it with a circumflex (^) anchor
- A pattern can be specified to match at the end of a string only by following the pattern with a dollar sign anchor
- `/^pearl/` matches "pearls are pretty" but does not match "My pearls are pretty"
- `/gold$/` matches "I like gold" but does not match "golden"

# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- Anchors do not match specific characters in the string; rather, they match positions
- When a circumflex appears in a pattern at a position other than the beginning of the pattern it has no special meaning. (It matches itself.)
- Likewise, if a dollar sign appears in a pattern at a position other than the end of the pattern, it has no special meaning



# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- **Modifiers:**
- They can be attached to patterns to change how they are used, thereby increasing their flexibility
- The modifiers are specified as letters just after the right delimiter of the pattern
- The **i** modifier makes the letters in the pattern match either uppercase or lowercase letters in the string
- **/Apple/i** matches 'APPLE', 'apple', 'APPlE', and any other combination of uppercase and lowercase spellings of the word "apple."

# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- The `x` modifier allows white space to appear in the pattern. Because comments are considered white space, this provides a way to include explanatory comments in the pattern

`/\d+ # The street number`

`\s # The space before the street name`

`[A-Z][a-z]+ # The street name`

`/x`



# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- Other Pattern-Matching Methods of String
- The replace method is used to replace substrings of the String object that match the given pattern.
- The replace method takes two parameters: the pattern and the replacement string.
- The **g** modifier can be attached to the pattern if the replacement is to be global in the string, in which case the replacement is done for every match in the string.
- The matched substrings of the string are made available through the predefined variables \$1, \$2, and so on

# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- `var str = "Fred, Freddie, and Frederica were siblings";`
- `str.replace(/Fre/g, "xyz");`
- str is set to "xyzd, xyzddie, and xyzderica were siblings", and \$1, \$2, and \$3 are all set to "Fre"



# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- The match method takes a single parameter: a pattern
- It returns an array of the results of the pattern-matching operation
- If the pattern has the g modifier, the returned array has all of the substrings of the string that matched.
- If the pattern does not include the g modifier, the returned array has the match as its first element, and the remainder of the array has the matches of parenthesized parts of the pattern if there are any

# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- `var str = "Having 4 apples is better than having 3 oranges";`
- `var matches = str.match(/\d/g);`
- `matches` is set to `[4, 3]`

- `var str = "I have 428 dollars, but I need 500";`
- `var matches = str.match(/(\d+)([^\d]+)(\d+)/);`

The following is the value of the `matches` array after this code is interpreted:

- `["428 dollars, but I need 500", "428", " dollars, but I need ", "500"]`



# PATTERN MATCHING BY USING REGULAR EXPRESSIONS

- The split method of String splits its object string into substrings on the basis of a given string or pattern.
- The substrings are returned in an array
- `var str = "grapes:apples:oranges";`
- `var fruit = str.split(":");`
- fruit is set to [grapes, apples, oranges].